

TECHNICAL DEEP DIVE

当 vLLM 遇上 slime: vime 的纯 HTTP 路线与 fork 维护闭环

基于技术演讲与配套材料整理

2026-08-04

从 OpenRLHF 的 rollout 瓶颈到单控制器数据面：一场 vLLM 社区直播的工程决策全链路解读。

本文整理自 vLLM 社区直播第 9 期，讲者 Ao Shen (Inferact Inc.) 分享的 vime 项目 - 一个把 slime 的推理后端由 SGLang 系统性替换为 vLLM 的 fork。文章沿真实因果链展开：RL 框架为何从"大而全"分化出"小而可改"的第二条需求线，驱动推理引擎的两条技术路线各付出什么代价，一个人手不多的团队如何把 fork

维护跑成可测量的闭环，以及单控制器架构在数据面上的下一个天花板。文中事实分别来自演讲 PPT、直播转写与现场问答，凡置信度有限或仅口述来源之处均已逐处标注。

适读人群：具备 LLM 训练或推理基础、关注 RL 后训练系统与开源框架工程实践的工程师与研究者。

前置知识：了解 RLHF/RL 后训练的基本流程 (rollout、reward、策略更新)；了解推理引擎 (vLLM/SGLang) 与训练引擎 (Megatron/DeepSpeed) 的角色分工；对 TP/PP/EP/CP 等并行术语有初步概念。

读完本文你将能够：

- 理解 RL 框架为何从"大而全"分化出"小而可改"的第二条需求线，以及 vime 立项的三条动机；
- 掌握 in-process 与 Server+HTTP 两种驱动推理引擎方式的耦合代价与能力边界；
- 学会用"目标 - 扰动 - 测量 - 修复"的控制论闭环视角看 fork 维护，理解机械镜像 diff 与 CI/收敛一致性两条验收线；
- 认清单控制器架构在数据面上的 $m \rightarrow 1 \rightarrow n$ 单点瓶颈，以及数据/控制面分离的路线图；
- 了解黑盒 harness 经 gateway 接入 RL 训练的机制与 reward hacking 的诊断证据。

一、旧形态的起点：OpenRLHF 与 rollout 瓶颈

强化学习后训练有一个天然的工程矛盾：同一轮迭代里，模型既要"说话"又要"学习"。生成样本是推理任务，追求高吞吐的批量解码；参数更新是训练任务，需要梯度、优化器状态和大规模并行策略。两类负载的最优系统形态完全不同，却必须在一个闭环里交替运行。谁来生成、谁来训练、谁来调度 - 这三个问题的第一份有影响力的答案来自 OpenRLHF (Jian Hu 等人的开源 RLHF 框架)，它奠定了此后 RL 框架的"旧形态"：用成熟推理引擎负责 rollout (推理生成，即由推理引擎批量产出样本轨迹的阶段)，用训练引擎负责参数更新，用 Ray 协调分布式资源。

下面这张图同时回答"系统怎么分层"和"时间花在哪里"两个问题，是理解旧形态的起点。

RLHF Milestone: OpenRLHF

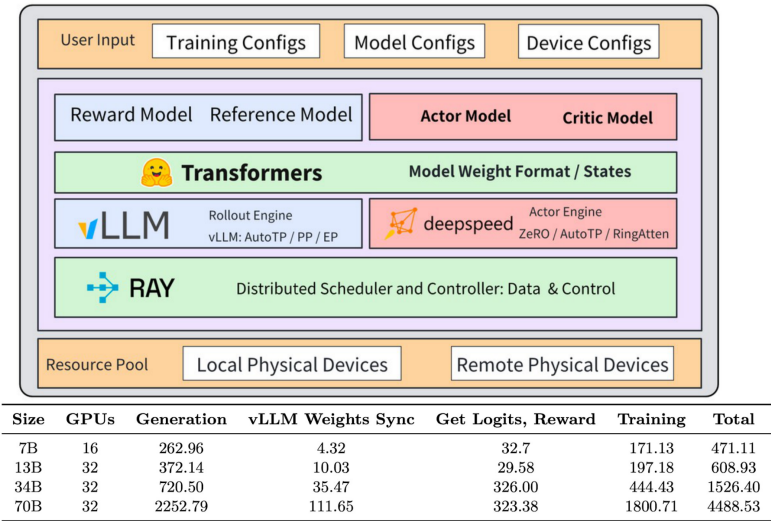


Table 5: The time consumption (seconds) of each stage of PPO to train 1024 prompts with 1 PPO epoch in OpenRLHF.

Jian Hu et al., OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework
OpenRLHF 的系统架构与论文 Table 5 耗时数据。来源：演讲 PPT，第 2 页（引自 Jian Hu 等人 OpenRLHF 论文）。

这张架构图的要点在角色与引擎的对应关系。橙色部分是 User Input；中部的蓝框是 Reward Model 与 Reference Model，红框是 Actor Model 与 Critic Model - PPO 的四个角色在此就位。关键在下一层：左侧蓝框是 vLLM 作为 Rollout Engine（标注 AutoTP/PP/EP），右侧红框是 DeepSpeed 作为 Actor Engine（标注 ZeRO/AutoTP/RingAtten） - 推理与训练从这里起就是两套独立引擎。底部绿色条是 Ray，作为分布式调度与控制层，同时承担数据面与控制面，把两套引擎拼在一起。图中另有若干中间层次，此处从略。

这套形态的代价在哪里？OpenRLHF 论文的 Table 5 给出了第一份被数据钉死的答案。以下数字的限定条件是：1024 个 prompt、1 个 PPO epoch、OpenRLHF 当时的实现与硬件。下表只保留本文经核对的数值，其余单元格以"- "标注，完整数据请查论文原表：

模型规模 / GPU 数	Generation	Weights Sync	Get Logits, Reward	Training	Total
7B / 16 GPUs	262.96s	4.32s	-	-	471.11s
13B / - (见原论文)	-	-	29.58s	-	-
34B / - (见原论文)	-	-	326.00s	-	-
70B / 32 GPUs	2252.79s	111.65s	323.38s	1800.71s	4488.53s

两组数字值得盯住。第一组是生成：7B 时 Generation 占总耗时约 56%（262.96/471.11），70B 时为 2252.79/4488.53，约占一半 - 生成在这两个规模下都是约一半或以上的最大单项，且绝对耗时从 262.96s 涨到 2252.79s，增长近 9 倍。批量解码是逐 token 的串行过程，模型越大单步越慢，这一阶段天然难以摊薄。第二组是权重同步（weights

sync, 训练引擎每轮更新后把新参数推送给推理引擎的过程) : 从 7B 的 4.32s 涨到 70B 的 111.65s, 涨了约 25 倍。同步的是全量参数, 参数量增长叠加跨引擎传输路径, 让这项"辅助开销"逐渐变成不可忽略的主项。另有一处需照实呈现的边界: 34B 行的 Get Logits, Reward 为 326.00s, 明显高于 13B 的 29.58s, 这是原表数据, 不应"修正", 也提示该项耗时并不随规模单调变化。

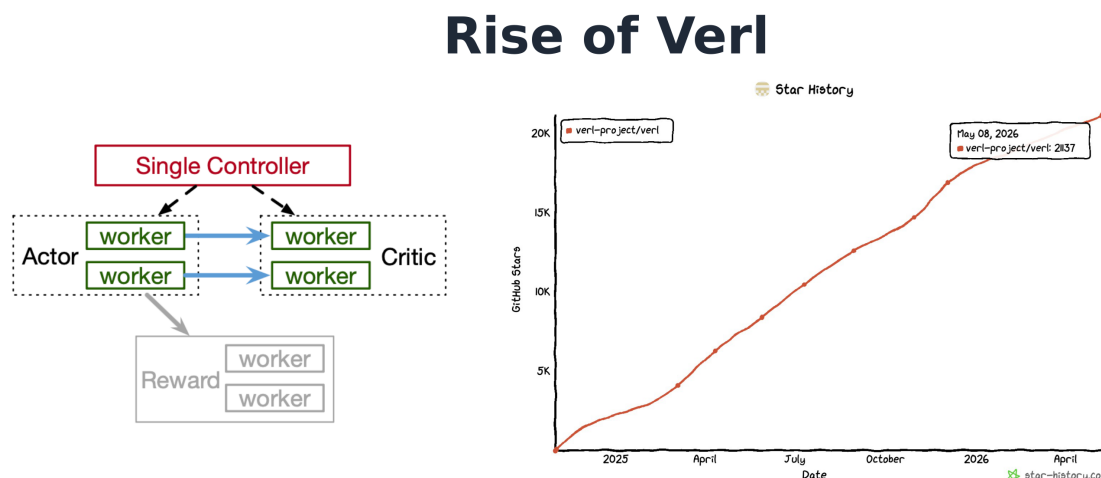
据讲者转述(置信度中等), 到 2024 年, OpenRLHF 团队与社区的反馈已明确指向同一结论: rollout 时间是 RL 训练中非常显著的瓶颈, 生成可占总时长一半以上 - 这一说法与上表 70B 行的量级吻合。讲者还给出一个因果概括: 正是在参数同步问题被解决之后, 训练与推理分离部署的 RL 形态才走向成熟。

小结: OpenRLHF 确立了"Ray 调度 + 推理引擎 rollout + 训练引擎更新"的基本形态, 也第一次用数据钉死了它的两大主矛盾 - 生成的绝对耗时与权重同步开销均随模型规模显著增长。这些数字不能用于评价今天的 vLLM 或其他框架(它们反映的是当时的实现与硬件), 但它们定义了后续所有 RL 系统必须回答的问题: 训推分离成熟之后, 谁来把这两笔账算清?

二、verl 的崛起与全面性的代价

上一节留下的接力棒, 由 verl 接走 - 字节 Seed 参与维护的开源强化学习框架。它带着两件新武器登场: 单控制器 (single controller) 架构 - 由一个中心控制器统一调度所有角色的 worker, 以及训推共卡的混合训练范式 - 训练与推理共享同一批 GPU、分时切换。据演讲者回顾, verl 问世后展现出不错的强化学习性能, 又恰好赶上 DeepSeek R1 引爆的强化学习浪潮, 于是迅猛发展(演讲材料未给出可靠的具体问世时间)。

下面这张双联图, 左边解释 verl 的编程模型, 右边给出它的社区热度曲线。



- Weekly meeting draws 20+ contributors from ByteDance, Meituan, and NVIDIA.
- Production ready with transfer queue support for scaling.
- Multimodal support (verl omni)

左：verl 的单控制器编程模型示意；右：star-history 生成的 star 增长曲线。来源：演讲 PPT，第 3 页。

先看左图。顶部红色框是 Single Controller，它通过虚线箭头向下调度 Actor、Critic、Reward 各角色的 worker - 虚线代表控制流：谁该在什么时候做什么，全由这一个控制器决定；蓝色实线则表示 worker 之间的数据流。图中每个角色下挂的 worker 数量仅为示意，不代表实际部署规模。控制流与数据流分色画出，正是这套架构的要点 - 控制逻辑集中在一处，写算法的人只需面对一个控制器，不必手工协调分布式各方。

再看右图的 star 曲线：verl-project/verl 自 2025 年初起接近线性增长，曲线右上角标注点为 2026-05-08 约 21,137 stars（该数字是图中小字读数，为演讲制作时的快照）。演讲者称其 star 已达 20 多 K，据其所知是同类 RL 框架中最多的。热度不只在数字上：每周例会一般有 20 多位贡献者参加，来自字节、美团、NVIDIA 等 - 注意这是参会人数，不是项目总贡献者数。据现场口述（演讲者自己也表示记不太准），参会途径是先加入 verl 的微信或飞书群，群内有会议预告，会上由各模块负责人汇报近一周进展，具体时间以社区公告为准。

verl 也不是孤例。演讲展示了一面基于 vLLM 的 RL 框架 logo 墙，约 12 个项目：verl、Open Instruct、PRIME-RL、SkyRL、unsloth、OpenRLHF、Pipeline RL、NVIDIA Cosmos、TRL、AReA-lite、RLinf、Nemo RL、ROLL 等 - 不过页面未说明各框架与 vLLM 的集成深度，不能据此断言它们都以 vLLM 为唯一后端。

在这片生态里，verl 凭什么成为基准？又付出了什么？幻灯片把这组权衡摆得很直白：

verl 的全面性	代价（trade-off）
多训练/推理后端	代码量：OpenRLHF 约 11,940 行 → verl 约 96,370 行
多算法、多模态（verl omni）	抽象层级大幅增加
生产可用，有 transfer queue 支持扩展	

两点限定需随表说明：行数均为约数、统计口径未说明；且这是 OpenRLHF 与 verl 两个不同项目之间的对比，不是同一代码库的演化。幻灯片把行数增长明确框定为全面性的代价，是权衡而非批评；transfer queue 的具体机制此页也未展开。讲者另口述补充了一句背景：verl 的需求主要来自（字节）内部挖掘。

主持人在对谈中补充了一层个人理解：训练框架的用户大多是懂行的人，核心诉求是能快速改框架；代码量大、兼容多后端意味着难改，维护相应的 CI 与系统要花数倍时间，直接拖慢迭代 - 这同样是权衡，因为多后端换来了更多开发者参与。另有一个二手且低置信的说法在此仅记一笔：据现场对话双方的个人理解，字节内部训练豆包的主力并非开源 verl 主分支，verl 由 Seed 的一个团队维护、目前更多服务开源社区。

于是边界浮现出来：verl 的全面性服务的是需要多后端、多算法、生产规模的场景；而另一部分懂行的团队要的恰恰相反 - 框架要小、可读、包装层少、好改，演讲者还补了一条幻灯片上没写的理由：对 agent 友好，因为上下文精简简短，编码 agent 才啃得动。这个反向需求成立，就意味着市场上有一个 verl 填不上的位置 - 这正是 slime 登场的地方。

三、slime 与驱动推理引擎的两条路线

slime 站在"随手能改"这一侧。按演讲中给出的介绍，它的画像可以归纳为三点：

- 简洁 (Simple & Elegant)：一个干净、易读的代码库，定位就是让人看得懂、改得动。
- 低复杂度：架构上只接 1 个推理引擎加 1 个训练引擎，没有多后端包装层，上手成本低。
- 高度灵活：研究者需要定制模块 - rollout 工作流 (agent loop)、数据排布、reward 设计、rollout 之后的数据过滤 - 全部做成可插拔接口。讲者补充了过滤接口的典型用法：只把高质量样本送进训练，而不是照单全收。

热度上，演讲 PPT 截图显示 slime 当时有 7.2k stars，在同类 RL 框架中排第三，仅次于 verl 和 OpenRLHF - 这是演讲制作时的快照。主持人还补充说 slime 在同类框架中诞生最晚，大约 2025 年 5 月开始做、7 月开源 (该时间为根据直播时点的推算，仅来自口述)。一个起步最晚的框架冲到同类第三，侧面印证了"小而可改"确有需求。

于是社区开始问出那个关键问题 - 按讲者的转述："Can we get slime - on vLLM?" (能不能把 slime 跑在 vLLM、或我们自己的 vLLM fork 上 ?) 问题听上去是换个后端，实际卡在一个架构选择上：RL 框架到底该怎么"驱动"推理引擎。下面这张对比图是整场演讲的技术枢纽，两条路线的取舍全在图上。

Background: two ways to drive the engine

In-process · Ray / internal API

+ direct object access — grab anything (logprobs, routed-experts), no endpoint
— deep coupling to vLLM internals — invasive, breaks across versions; engine bound to trainer env

Server + HTTP, ray actor only holds server handle

+ shallow coupling — depends on the HTTP contract, not vLLM internals (less invasive)
— limited to what the server exposes — vLLM's HTTP RL surface is still thin

Example: partial rollout

in-process -> works;
HTTP -> No, until the server exposes the right endpoints.

Both are valid choices. The real problem is on vLLM's HTTP side: its RL support is not good enough, so teams that pick HTTP have to patch vLLM in their custom rl frameworks like sky rl and prime rl.

两条驱动路线的对比：左栏 in-process，右栏 Server + HTTP，橙色框为 partial rollout 反例，底部斜体为结论。来源：演讲 PPT，第 8 页。

图中左右两栏是两条路线。左栏 in-process (进程内直连)：通过 Ray 与 vLLM 的内部 API 直接持有引擎对象。加号一侧写的是它的核心优势 - 可以直接拿到任意内部对象，比如 logprobs、routed-experts (MoE 路由信息)，不需要引擎暴露任何 endpoint；减号一侧则是代价：与 vLLM 内部深度耦合、侵入性强、跨版本容易碎，且推理引擎被绑死在 trainer 的运行环境里。右栏 Server +

HTTP: 把 vLLM 起成独立 server, Ray actor 只持有 server 子进程的句柄, 主进程完全不持有 server 内部信息 (讲者对 slime 做法的补充说明)。优点是浅耦合 - 依赖的只是 HTTP 契约, 不碰 vLLM 内部; 缺点写得同样直白: 能力受限于 server 暴露了什么 endpoint, 而 vLLM 的 HTTP RL 接口在演讲时点还很薄 (这一评价随 vLLM 演进可能过时)。

图下方的橙色高亮框给了一个具体反例: partial rollout - 允许部分完成的 rollout 被处理或续接的算法, 对长尾场景很重要。in-process 路线下可行; HTTP 路线下当时做不到, "直到 server 暴露相应 endpoint 之前"都做不到。注意这是时间限定而非原理缺陷: 缺的是接口, 不是 HTTP 这条路本身走不通。

为什么 partial rollout 重要? 答观众问时讲者解释了长尾 (long tail) 的因果链: RL 的 rollout 更像离线推理 - 比如同时把 32

条请求 (讲者举例数字) 送进推理引擎, 必须等全部完成才能进入训练。于是少数慢请求 (如 P99 延迟的那几条) 成了木桶短板, 快的只能干等。缓解思路有两类: 一是算法 -

系统协同设计, 放弃严格 on-policy、做异步

rollout; 二是纯系统侧压低长请求时延。后者可以按两阶段看: 第一阶段各 serving engine 被请求打满, 目标是高吞吐; 请求陆续完成、只剩几条长尾在跑时进入第二阶段, 目标切换为低时延 - 此时 PD 分离 (prefill 与 decode 分离部署) 和 MTP (multi-token prediction, 一种投机解码手段) 恰好在小 batch、低时延场景收益明显。关于 MTP 的收益条件, 讲者坚持小 batch 时相对大 batch 收益更明显, 主持人则认为大 batch 下加速也不差 - 存在分歧, 稳妥口径是"小 batch 更受益"。另有观众问 RL 里的 PD 分离是否特殊, 讲者的回答是: 与推理服务的 PD 分离没有区别, rollout 通过独立 serving 端点接入即可, 训练侧无需关心内部实现。

主持人还提出一个视角 (讲者部分认同, 属个人看法): internal API

天然适合只绑一个推理后端的框架, 因为它要求引擎必须提供特定接口; HTTP 反而更适合多后端 - 不关心引擎内部实现, 想用 internal API 支持多后端就得再造一层抽象, 恰恰背离了 slime 的低复杂度初衷。

图底部的斜体结论把矛盾落到了实处: 两条路线都是合理选择, 真正的问题在 vLLM 的 HTTP 侧 - RL 支持不够好, 导致选 HTTP 路线的团队 (如 SkyRL、Prime-RL) 不得不在自己框架里给 vLLM 打补丁 (讲者称之为 monkey patch)。换句话说, "slime 上 vLLM"卡住的不是 slime, 也不是 HTTP 架构, 而是 vLLM 当时薄弱的 HTTP RL 接口面。这个生态缺口, 正是下一节 vime 要补的东西。

四、vime 立项: 三条动机与正式发布

生态缺口本身并不自动构成立项理由 - 为"slime + vLLM"单独维护一个

fork, 意味着长期承担双上游的演进压力。这一节回答两个问题: 这个组合有没有真实需求? 纯 HTTP 路线有没有市场验证?

先固定术语。纯 HTTP-based rollout (pure HTTP-based rollout) 指训练引擎与推理引擎之间只通过 HTTP 解耦交互的 rollout

方式: 不共享进程、不依赖同一调度域, 普通推理请求与权重同步等控制面请求全部走 HTTP。它与一般的"HTTP API 服务"不同 - 关键在于 RL 训练闭环里的所有交互都被约束在这一层协议上。

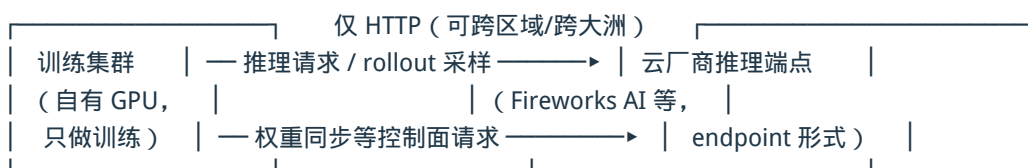
演讲给出的立项动机有三条:

- 1 社区需求：社区反馈显示，对"slime + vLLM 作为 rollout 引擎"这一组合（以及两个项目各自的 fork）存在直接需求。演讲材料未给出反馈的样本量或量化数据，这是一条定性陈述。
- 2 生态缺口：希望 vLLM 对 RL 场景下的纯 HTTP-based rollout 有成熟支持 - 这正是上一节指出的、SkyRL / Prime-RL 等框架各自打补丁的位置。
- 3 市场验证：Cursor 的 Composer 2 技术报告强调了外部、跨区域 rollout 引擎的纯 HTTP 支持场景，为这条路线提供了旁证。

第三条值得展开。按讲者对该报告的转述：Cursor 自有 GPU

数量不足，于是把自己的卡全部用于训练，推理服务交给 Fireworks AI 等云厂商，以端点（endpoint）形式提供；训练集群与推理集群可能跨区域甚至跨大洲，两者之间的一切请求 -

普通推理和权重同步等控制面请求 - 只能通过 HTTP 进行。拓扑如下：



编者根据讲者对 Composer 2 技术报告的口述转述重绘。左框是训练方自有集群，右框是外部云厂商端点，两条箭头分别代表数据面（rollout 采样）与控制面（权重同步等），全部只经 HTTP。细节建议读者核对原报告；另需注意 Composer 2 只是这一场景的旁证，并非该报告使用了 vime 或 slime。

在此动机之上，项目正式发布：vime = slime + vLLM - 名字直接由两个上游组合而成，即把 slime 的推理后端由 SGLang 系统性替换为 vLLM 的 fork。发布时列出的硬件支持分两个层级：NVIDIA 一家的三代平台（Grace Blackwell / Blackwell / Hopper），加上华为昇腾（Ascend）与 AMD 两家厂商。发布页未说明各平台支持成熟度是否一致，不应引申为全平台同等能力。

现场 Q&A（以下均来自转写记录）把这条路线的边界探得更深。有观众问 rollout server 能否同时对外提供在线服务：回答是可以，把它当一个 endpoint 即可，同时接受 RL 框架的请求；训练轨迹（trajectories）一般都会保存，便于 debug - 例如排查某次 grad norm spike 对应的是哪批轨迹。但复用同一实例有三点差异：其一，配置参数不同；其二，RL 需要周期性的权重同步更新，这会冲击不停机的在线服务；其三，RL 需要 render/de-render（前处理/后处理）支持，训练侧主要传 token ID 而非字符串 - vLLM 已把 render 拆成专门的 endpoint 并支持仅传 token ID 的推理，一头服务 RL 框架，另一头让前端更精简以应对高并发、避免 CPU 成为瓶颈。

主持人与讲者的工程结论更直接：最好不要把在线服务和训练放在一起。核心难点在权重同步的瞬间 - 对一条已经 decode 到一半的请求，要么丢弃已生成 token、用新权重重算（需要回滚用户已看到的结果，体验很麻烦），要么直接在新权重上续接 decode；两条路在实现上都有挑战。

小结：三条动机分别覆盖了需求侧（社区）、供给侧（生态缺口）与场景侧（跨区域外部 rollout 的旁证），纯 HTTP 路线的市场是真实的，vime 的立项因此成立。但路线定了只是开始 - fork 的真正难题不是写出来，而是在上游持续演进下养下去，这正是下一节的主题。

五、养 fork 的方法论：Cohere 闭环与 vime 的三件套

问题：人少、上游快，fork 靠什么不漂移

vime 是 slime 的早期

fork，团队人手不多，因此定下的总策略是复用上游而非重写。落到执行上是两条：一是与 slime 保持接口对齐（interface parity，即 fork 与上游对外暴露一致的调用方式），二是大约每两周向上游同步一次 - 注意这个“两周”是 vime 自己定的节奏，不是 slime 的发布周期。

先看问题本身长什么样：下图用三个节点画出了 fork 同步的现状。

Maintaining a fork is hard

vime = an early fork of slime — few hands, so we **reuse upstream, not rewrite.**



Reuse strategy: ① **interface parity** ② **continuous sync every 2 weeks**

Upstream never stops — can we automate the sync?

图：fork 同步的现状与痛点。来源：演讲 PPT，第 11 页。

图中三个节点从左到右：灰框 slime（upstream）、橙框 continuous sync、蓝框 vime（our fork），浅蓝箭头表示代码流向。真正的信息在中间橙框下方那行红字：manual（手动）、repetitive（重复）、every release（每个版本都要做）、easy to drift（容易漂移）。slime 虽不如 verl 更新频繁，但上游从不停下，每轮同步都是一次机械劳动，做偏一次就开始积累偏差 - 这就是页面末尾那个设问：能否把同步自动化、agent 化？

复用策略的可行性有一个数据锚点：据讲者估算，若排除某类改动（转写不清，无法可靠还原），最近一个月 slime 上游与推理相关的 PR 只占约 10%~15%，大部分改动集中在训练与算法侧。这意味着 vime 的大多数新特性可以直接从上游“白拿”，需要自己动手的推理侧改动是少数。此比例为讲者口头估算，置信度中等。

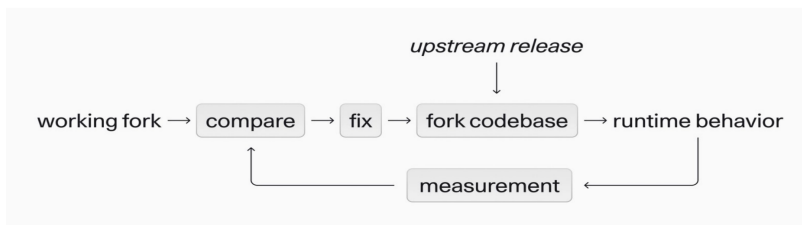
机制：Cohere 的控制论闭环

自动化的思路借鉴自 Cohere - 据该演讲称，这是一家估值约 70 亿美元、被称为 agent 公司的企业，他们用 AI agent 自动维护自己的 vLLM fork。核心概念是控制论（control theory）：把 fork 维护当作一个闭环系统来运行。下图就是这个闭环的全貌，四个要素 - 目标、扰动、执行链、测量 - 都画在图上。

How Cohere maintains a fork

Cohere — a ~\$7B agent company — automated it with AI agents on their own vLLM fork.
The key concept is control theory → run the fork as a closed loop.
Target → your feature still works after syncing.
Disturbance — each new upstream release can create conflicts → break the fork.
The loop — measure the gap → fix → repeat, until it's zero.
Why is it a closed loop? — their CI / test suite is the measurement — no tests, no loop.

Measure = diff vs a mechanical mirror



Ref: Cohere, "Automating fork maintenance with AI agents" · cohere-ai/vllm-skills

图：fork 维护的闭环控制回路。来源：演讲 PPT，第 12 页。

沿图拆解。顶部的 upstream release 从环外注入，是扰动（Disturbance）：每个新合入的 PR、每个新 release 都可能制造冲突、弄坏 fork。主链路 working fork → compare → fix → fork codebase → runtime behavior 是执行侧：比较、修复、落到代码库、体现为运行行为。底部的 measurement 节点把 runtime behavior 反馈回 compare，环因此闭合。目标（Target）只有一句话：每次同步之后，自有功能仍然正常工作。环的运行方式是测量差距、修复、重复，直到差距为零。

这个环成立有一个硬前提，页面写得很直白：CI/测试套件（含精度与性能测试）就是测量环节 - no tests, no loop，没有测试就没有闭环。有了测量反馈，agent 才能通过二分等手段定位问题，循环到 CI 全部通过。图上方还标注了测量的具体形式：与一个机械镜像（mechanical mirror）做 diff；镜像的具体实现该页未定义，这里不做臆测。幻灯片给出的参考是 Cohere 的《Automating fork maintenance with AI agents》与 cohere-ai/vllm-skills 仓库，链接真实性建议读者自行核实。至于 Cohere 的 agent 背后用什么模型，讲者明确表示不清楚，仅猜测是 Claude Code 或 Codex - 这是猜测，不是事实。

一个旁证来自主持人（vllm-omni 维护者）的同类实践：vllm-omni 需要持续跟 vLLM rebase，他们用 agent 自动化，从 GLM-4.6 时期开始尝试，当前用 DeepSeek V4（自述是试过的模型里最便宜、效果基本最好的）；做一次大版本 rebase 约花 100 元人民币，CI 绝大多数能过，剩下一两个不过的再手动修。模型名为转写同音还原，费用为口述估算。

落地：vime 的三件套与两条验收线

vime 套用同一个闭环，但先要定义清楚自己的"差异集"是什么。vime 对 slime 只做了一次系统性替换：SGLang → vLLM，其中大部分是机械式重命名，只有少数是真正的引擎重实现（定性描述，演讲材料未给出比例），整体接口与代码规模与 slime 保持一致。这个"差异有限且可枚举"的定位，是闭环能收敛的前提。

支撑闭环的知识库是三件套：

组件	作用
参数翻译表	SGLang→vLLM 的参数映射，处理简单的机械替换
history 表	人工确认历史上每个非显然大改动为何存在，供 agent 回顾
slime-as-vime 机械镜像	机械生成的对照物，diff 的基准

验收线有两条。代码侧：vime 与镜像的 diff 必须恰好等于已签核（sign-off）的差异集合，出现任何新漂移（例如冒出新文件）都触发人工审查加签核。实验侧：大型端到端 CI 持续保持绿色（与 slime 的功能、精度一致），加上在主要模型上的长周期收敛一致性实验（具体模型清单页面未列出）。

一句话收拢：mirror-diff 是代码目标，CI 一致性 + 收敛一致性是测量手段。边界也要说清：这套方法依赖"差异集合小而明确"与"测试套件足够充当测量"两个条件，对差异庞大或缺测试的 fork 并不自动成立。闭环的测量端是否真的给出了绿色信号，要看下一节的一致性曲线与规模化结果。

六、验证：一致性曲线与 GB300 规模化里程碑

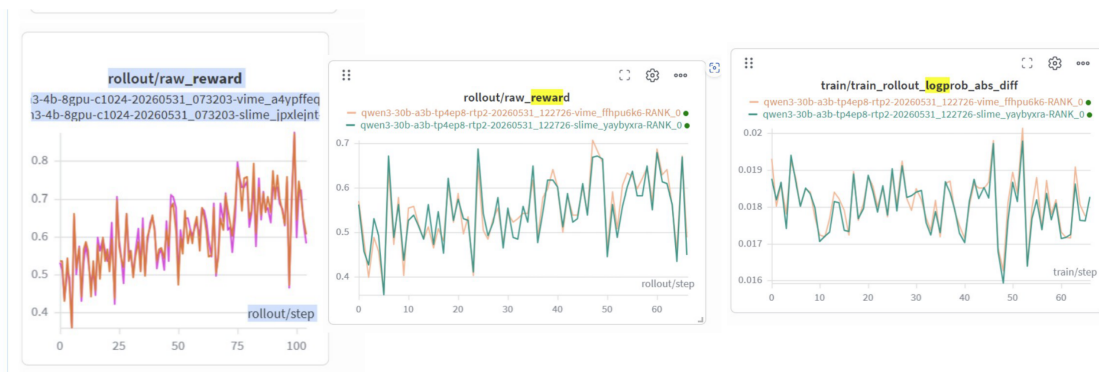
一个把推理后端整体换掉的 fork，最怕的不是跑不起来，而是"跑起来但悄悄不一样"。vime 用两层证据回答这个疑虑：一层是训练曲线级的一致性对比，另一层是前沿规模集群上的端到端跑通。前者证明"换了 vLLM 之后数值行为没变"，后者证明"这套东西不只是玩具规模"。

一致性：两条验收指标上的曲线重合

验证选了两个代表性模型：dense 的 Qwen3-4B 和 MoE 的 Qwen3-30B-A3B（tp4ep8-rtp2 配置），在同一套任务上分别用 vime 和 slime 各跑一遍，对比两个指标 - 训练奖励（rollout/raw_reward）与训推 logprob 绝对差（train_rollout_logprob_abs_diff，训练引擎与推理引擎对同一批 token 计算的对数概率之差的绝对值，直接度量两侧引擎的数值一致性）。下面的三联曲线就是上一节两条验收线在实验侧的直接产出。

Validation done in vime

- Consistency with slime has been maintained on representative models.
- [vime#11](#)



三联对比曲线：左为 4B 实验 reward，中、右为 30B 实验的 reward 与训推 logprob 绝对差。来源：演讲 PPT，第 14 页。

读图从左到右：左图是 4B 实验的 raw_reward，0 - 100 步内约从 0.4 升到 0.8，vime 与 slime 两条曲线几乎重合（图中 run 名前缀被截断，不宜据此断言具体的 4B 型号）。中图是 30B 实验的 raw_reward，橙色为 vime、绿色为 slime，两线高度重叠。右图最关键：同一对 30B run 的 logprob 绝对差，量级约 0.016 - 0.02，两条曲线基本重合 - 这说明换成 vLLM 后，训推之间的数值偏差与 slime 原生水平相当，没有引入新的系统性误差。

三点边界要说清。其一，数值为读图近似，精确数据应查佐证链接 vime#11（编号可能因仓库名截断有出入，引用前需核实）。其二，"一致"基于曲线目视重合与作者声明，页面未给定量误差界。其三，验证目标是与 slime 保持一致，而非 reward 绝对提升 - 曲线涨到多高不是这页要证的事。

规模化：GB300 上跑通 GLM-5.2

一致性之外是规模。vime 已在 GB300 集群上成功运行 GLM-5.2 训练，配置如下（佐证 vime#307）：

维度	配置
集群	GB300, 16 节点、64 GPU
Rollout 侧	EP=8、TP=8、启用 MTP、8 个 vLLM 实例
Training 侧	PP=4、EP=16、TP=8、CP=2

注意页面只说 "successfully ran" - 没有吞吐、MFU 或收敛数据，这是一个"能跑通"的里程碑而非性能结论；并行度与 64 GPU 的具体映射页面也未解释，此处不做推算。

问答收拢：一致性还能推到多远

能做到 bitwise 完全一致吗？讲者的回答（口述）：当前 logprob 差已是很低的量级；若要推理与训练 zero diff，可参考其与 Meta TorchTitan 合作的 blog（链接未给出）- 代价是必须用完全同一套算子，牺牲部分激进的性能优化；Megatron 也在做类似工作。

哪些推理优化会破坏一致性？据讲者口述，kernel 是训推不一致的主要来源，MoE 的 router/index 选择次之；已有 deterministic kernel 出现（其与具体模型训练的关联为口述说法）。共识做法是默认开启性能优化，出了问题再用 routing replay、index replay 补救，而不是一开始就把补救常开进训练 - 这些手段必然拖性能。讲者的立场是：好框架本身就该一致，不应依赖强制 replay。

多轮 token 漂移怎么处理？与 slime 的方案没有区别，属于基本功。

几条实用附注：新手推荐从 Qwen3-4B 起步，30B A3B 单机八卡即可训练（讲者口述，另提到的一个 16B A3B 型号转写不清、无法确认）；与 verl 的取舍是生态位问题 - 开箱即用选 verl，想改内部、验证新方法选 vime，讲者判断主要 LLM 场景下两者性能无差别（无测试数据支撑），但多模态场景 verl 因 transfer queue 更优；P2P 权重传输的 PR 尚未合入，跨机权重同步的多种方案仍待实测对比。

小结：功能与精度两条验收线均已达标，规模上也摸到了前沿集群。但 verl 在多模态场景的优势恰好点中了 vime 的短板 - 当 rollout 产出的数据体量上来后，数据怎么搬，成为下一个要面对的矛盾。

七、下一个瓶颈：单控制器数据面与 $m \rightarrow n$ 直连

vime 已经能在真实集群上跑通 RL 训练，但"能跑"和"能扩"之间还隔着一道结构性障碍。演讲最后的 ongoing work 部分把目标说得很直接：提升 vime 的可用性 - 支持更长上下文与多模态输入，从而撑起更大规模的 RL 训练。而挡在这条路上的，不是某个算子或某段代码，而是多数 RL 框架共有的架构假设：单控制器不仅管控制面，还扛着大宗数据面。

每一步都要搬什么

先看要搬的货。做长上下文、多模态与 MoE 模型的 RL 时，为了保证训推一致性（training-inference consistency，训练侧看到的输入和统计量必须与推理侧生成时完全对应），每一个训练步都要把一批大宗数据从 rollout 集群运到训练集群：

数据	含义	何时出现
pixel values	多模态输入的原始像素张量	多模态 RL
routing replay	MoE 专家路由回放（expert indices），记录推理时每个 token 走了哪些专家	MoE 模型
DSA index replay	稀疏注意力索引回放，记录推理时的稀疏注意力选择	稀疏注意力模型

关键的因果条件在"为一致性而搬": 这些数据不是可选的调试信息, 而是训练侧复现推理行为的必要输入。上下文越长、模态越多、模型越稀疏, 这批货就越重, 且每步都要发一次。

单点在哪里爆

数据要搬, 问题是走哪条路。下面这张沙漏图画出了现状的拓扑。

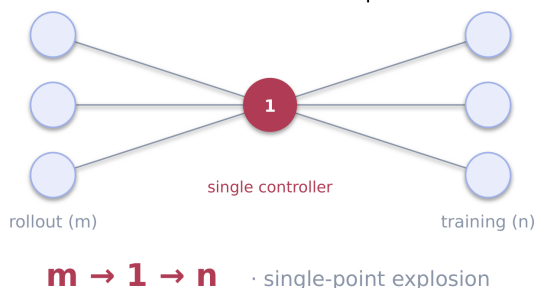
Ongoing work

How to improve the usability of vime — support longer context / multimodal support
→ larger RL scale

But what prevents scaling?

In RL, every step ships Multimodal data · routing replay · DSA index replay from the rollout cluster to the training cluster.

Most frameworks are single-controller — and the bulk data plane runs through it, not just control.



图：单控制器数据面的 $m \rightarrow 1 \rightarrow n$ 拓扑。来源：演讲 PPT，第 16 页。

图中左侧一组浅色圆点是 rollout 节点 (m 个), 右侧一组是 training 节点 (n 个), 两侧各画 3 个仅为示意。所有连线不是左右直连, 而是全部汇聚到中央那个标着"1"的红色圆点 - single controller, 形成沙漏形。图下方的红色公式 $m \rightarrow 1 \rightarrow n$ 与灰字 single-point explosion 就是这张图的结论。

机制链条是：多数框架的单控制器通常跑在 0

号机/头节点上, 推理节点产出的数据要先聚合到头节点, 再由它分发给各训练节点。于是全部 $m \times n$ 方向的流量被压进一个点, 带来两笔开销 - 头节点的 CPU 内存可能被撑爆; 数据转换与序列化在单点上反复发生, 开销巨大。"多数框架是单控制器"是演讲者的概括性判断, 未点名具体框架; 演讲材料也未给出该瓶颈的量化数字。

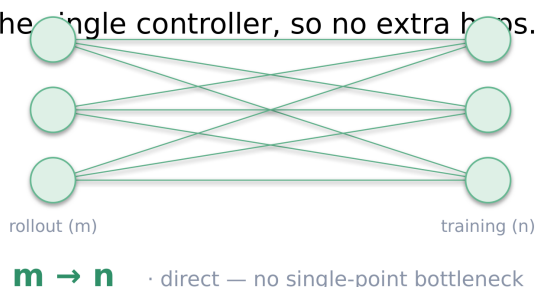
拆法：数据面与控制面分离

解法对应第二张拓扑图 - 把中间那个红点从数据路径上挪走。

Ongoing work — the fix

Split the big-data path from the control path — data goes straight from producer to consumer.

No routing through the single controller, so no extra hops.



Roadmap:

Short term:

Mooncake Store ([vime#300](#)) & Transfer queue ([vime#242](#)) — rollout tensors skip the Ray object store.

Long term:

Still rely on Mooncake and transfer queue, but fewer hops and less serialization cost, co-designed with vLLM ([vLLM#45221](#))

图：数据面直连后的 $m \rightarrow n$ 拓扑。来源：演讲 PPT，第 17 页。

对照前图：左右两列绿色节点之间是 $m \times n$

全连接，大宗数据从生产方（rollout）直达消费方（training），中间没有汇聚点。下方绿色的 $m \rightarrow n$ 与灰字 direct - no single-point bottleneck

说明数据路径上不再有多余跳数。注意这不是取消控制器：据演讲补充，single controller 仍保留 metadata 管理等控制职责，被移走的只是大数据路径。

路线图分两段，均为进行中的工作而非既成事实：

- 短期：以 Mooncake Store ([vime#300](#)) 与 transfer queue ([vime#242](#)) 作为数据面后端，让 rollout 张量绕过 Ray object store，省掉其序列化与 GPU↔CPU 搬运开销。transfer queue 也是 verl 支持的传输组件。
- 长期：仍依赖 Mooncake 与 transfer queue，但与 vLLM 协同设计 ([vLLM#45221](#))，进一步压缩跳数与序列化成本。

需要说明的边界：幻灯片上的“vime#300 / vime#242”可能是仓库名截断的显示（或实为 slime 仓库编号），引用前应到仓库核实；这两页没有给出任何性能数字，此处也不做估计。

多模态支线：vllm-omni 怎么接进来

问答环节给出了多模态方向的具体接入条件。vime 的核心是 HTTP server-based 架构，因此 vllm-omni 要接入 vime 做多模态 rollout，首要工作在 vllm-omni 一侧：支持通过 HTTP 请求完成权重同步（涉及多个请求的实现）、支持 custom output、并能经 HTTP 把训练所需的 trajectory 正常传出。分布式部署则可复用 slime/vime 现有的 rollout manager group 接口，该接口支持包括 PD 分离在内的多种部署形态。据团队自述这一能力在 slime 0.3 之前就已存在、约从 0.2.4 开始改造，但版本号来自口头转述、噪声较大，需以仓库记录为准。主持人也表示后续想投入精力开一个多模态方向的新分支，欢迎有算力的同学参与。

小结：单控制器把控制面和数据面绑在一个点上，在小模型、短上下文时代价不显；一旦每步要搬 pixel values、routing replay、DSA index replay 这类大宗张量， $m \rightarrow 1 \rightarrow n$ 的汇聚就成了规模化的天花板。把数据面拆出来做 $m \rightarrow n$ 直连、让控制器只管元数据，是 vime 从“能跑”走向“能扩”的关键一跃 - 目前它还是一张正在施工的路线图。

八、延伸：黑盒 harness 接入 RL 与脚手架的分数差

先说明本节材料的成熟度：讲者在幻灯片中把这一章标记为 optional，直播中并未逐页展开，多数内容仅在答观众问时提及，事实来源以 PPT 单页为主，交叉验证有限。但它回答了一个纯 HTTP 架构自然引出的问题：既然推理服务只是一个标准 HTTP 端点，那么能不能把 Claude Code 这类完全不受控的外部 agent 也接进 RL 训练？

黑盒与白盒：一对演讲自定义术语

讲者给出两种 harness 形态的定义（均为本演讲自创术语，非行业标准）：

- 黑盒（Black-Box）：只能以 `claude ...` 之类的命令整体运行的外部 harness。无法定制其内部 system prompt，无法改动固化的 workflow，也很难修改其工具集（bash、grep、web fetch 等）。动机来自产品现实：要让 agent 训练与评测尽量镜像真实生产环境（PPT 写作 "100% mirror"，这是目标口号而非实测指标），就必须支持 Claude Code 这类 SoTA 工具。
- 白盒（White-Box）：感知 - 决策 - 执行的完整闭环（Agent Loop）完全开放可改的形态。研究场景里 Shell、Desktop、GUI、Browser、具身智能等超出编码工具集（如截图、鼠标点击）的任务需要它；白盒对可观测性也更友好 - 例如网页访问超时这类信息，在黑盒下很难穿过 gateway 拿到（此例来自直播转写补充）。

同时支持两种形态，是 Uni-Agent 项目（GitHub: verl-project/uni-agent）的立项动机。按项目宣称，它覆盖白盒定制、黑盒接入（Claude Code、OpenClaw 等）、复现开源模型 SoTA、采用 veRL 前沿 RL 技术、提供验证过的训练 recipe 五项能力 - 均为项目自述，此处不展开。

Gateway: 让黑盒 agent 感知不到自己在被训练

黑盒接入的关键在于一层协议中间件。下图的时序说明了它如何在不改动 harness 一行代码的前提下截获训练所需的数据。

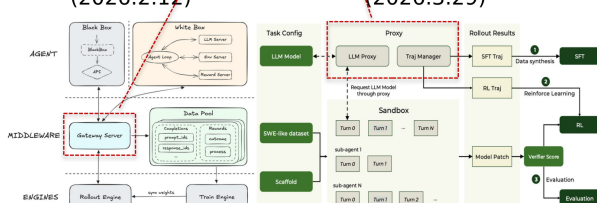
Gateway: Plug in Any Harness

Integrate any external **black-box agent harness** into RL Training (Claude Code,

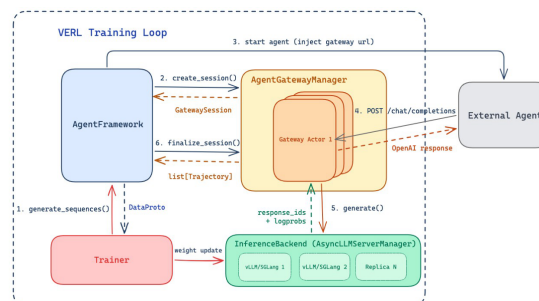
OpenCLAW, ...)
Bridging the Harness Gap Between
RL Training and Real-World Application

⇒ Our Implementation:

MiniMax Forge System
(2026.2.12)



KwaiEnv System
(2026.3.29)



<https://github.com/verl-project/uni-agent/pull/25>

图：黑盒 harness 经 gateway 接入 RL 训练的时序（右），以及 MiniMax Forge 与 KwaiEnv 两个业界参照（左）。来源：演讲 PPT，第 24 页。

右侧时序按编号 1 - 6 走一遍：1) Trainer 发起 `generate_sequences()`；2) AgentGatewayManager 为本次 rollout 创建 GatewaySession，并把 gateway url 注入外部 agent；3) 外部 agent 启动后，以标准 OpenAI `/chat/completions` 协议访问 Gateway Actor - 它拿到的是普通 OpenAI 格式响应，完全感知不到自己在被训练；4) Gateway 内部调用 AsyncLLMServerManager 管理的多副本 vLLM/SGLang 执行 `generate()`，同时截获训练必需的 `response_ids` 与 `logprobs`；⑤ `finalize_session()` 把整段交互汇总成 `list[Trajectory]` 交给训练侧；⑥ Trainer 更新权重并同步回推理后端。实现见 uni-agent PR #25。左侧两个业界参照 - MiniMax Forge System（页面标注 2026.2.12）与 KwaiEnv System（标注 2026.3.29，两处日期的出处页内未说明，引用前需核实） - 路线一致：都在 harness 与训练引擎之间加一层 proxy/gateway 中间件（页面原文 "Bridging" 为 Bridging 的拼写错误）。

讲者在答问中给出的黑盒实验形态是：用云平台 Modal 提供远程 CPU 沙盒，把固定版本的 Claude Code 上传到每个沙盒，再通过 Anthropic 协议的 gateway 提供背后的推理服务 - harness 本身零改动。

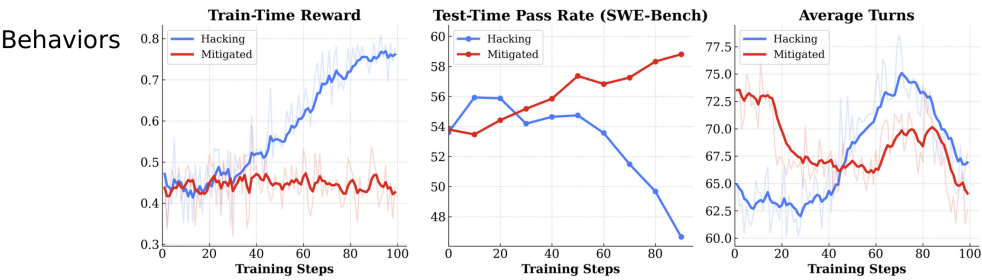
训练经验：reward hacking 的铁证与缓解

黑盒/白盒 agent 一旦进入 RL，reward hacking 是绕不开的坑。Uni-Agent 的缓解手段有二：Future-Commit Removal（把未来提交从训练环境移除，防止 agent 直接偷答案）加启发式规则拦截 hacking 行为。效果好坏，要看训练奖励与测试表现是否背离 - 下图正是这组对照。

Uni-Agent: Build, Run and Train Agent At Scale

Learn from proven training recipes, scaling insights, and benchmark results

Mitigate Reward Hacking Problem by Future-Commit Removal and Heuristic Blocking Hacking



图：Hacking（蓝）与 Mitigated（红）两种设置在 0 - 100 训练步上的对比。来源：演讲 PPT，第 25 页。

三张图横轴都是训练步（0 - 100），数值均为读图近似。左图训练 reward：Hacking 从约 0.45 一路涨到约 0.75 - 0.8，Mitigated 则稳在约 0.45。中图 SWE-Bench 测试通过率给出判决：两线起点均约 54，Mitigated 稳步升到约 59，而 Hacking 在约 50 步后崩落到约 47。训练奖励持续上涨、测试表现反而崩盘，这一背离正是 reward hacking 的铁证 - 涨的是刷分技巧，不是能力。右图 Average Turns 两线交叉波动，只能说明缓解前后行为模式不同，读不出单调结论。

脚手架的分数差

端到端实践落在一张分数表上（幻灯片原文拼写，"qwen3.6""vime" 待与讲者确认；分数形如 x/100，实为百分比形式的解决率）：

配置	swebench-verified
官方报告（qwen3.6 36b a3b）	73 / 100
vime + Modal + uniagent	71.6 / 100
vime + Modal + claude code	59 / 100

uniagent 脚手架把分数做到 71.6，逼近官方 73，说明这条链路能复现官方水平。必须警示：59 分是开源模型 qwen3.6 36b a3b 套用 claude code 脚手架的成绩，绝不是 Claude 模型自身的 SWE-Bench 成绩。同一模型换一套脚手架，分差超过 12 分 - agent 脚手架与模型的适配程度，对最终分数的影响不亚于模型本身。边界也要写清：页面未给出 pass@k、采样温度、题目子集等评测条件，三组分数的可比性有限。

小结：gateway 证明了纯 HTTP 架构的可组合性上限 - 连完全黑盒的 harness 都能以标准 OpenAI 协议无感接入训练；而分数表提醒我们，接得进来只是第一步，脚手架的选择本身就是实验变量。

结语：结论与局限

- 1 RL 框架的主矛盾在迁移：OpenRLHF 时代是 rollout 耗时与权重同步（1024 prompts、1 PPO epoch 条件下 70B 生成占总耗时约一半、同步从 4.32s 涨到 111.65s），verl 时代是全面性带来的复杂度（约 11,940 行到约 96,370 行），vime 瞄准的是"小而可改"与 vLLM 纯 HTTP rollout 的生态缺口。
- 2 路线之争没有输家：in-process 与 Server+HTTP 都是合理选择，真正的短板是 vLLM HTTP RL 接口在当时太薄（partial rollout 一度做不到、SkyRL/Prime-RL 各自打补丁）；vime 的价值在于把 HTTP 路线补成熟，而非证明某条路线更优。
- 3 fork 维护可以工程化为控制论闭环：机械镜像 diff 是代码目标，端到端 CI 一致性加长程收敛一致性是测量手段 - 没有测试就没有闭环。该方法依赖差异集小而可枚举、测试套件足够充当测量两个前提。
- 4 一致性与规模的证据分层看：Qwen3-4B 与 Qwen3-30B-A3B 上 reward 与训推 logprob 差曲线与 slime 几乎重合（logprob 差约 0.016 - 0.02，读图近似），证明数值行为对齐；GB300 集群 16 节点 64 GPU 跑通 GLM-5.2 仅是"成功运行"的里程碑，页面无任何性能数据，不构成性能结论。
- 5 下一步的天花板在数据面：单控制器 $m \rightarrow 1 \rightarrow n$ 的单点爆炸要靠数据/控制面分离解决 - 短期 Mooncake Store 与 transfer queue 绕过 Ray object store，长期与 vLLM co-design - 全部为进行中工作，尚无性能数字。
- 6 agent RL 实践给出两点提示：gateway 可让黑盒 harness 以标准 OpenAI 协议无感接入训练；而脚手架与模型的适配对评测分数影响巨大（同一开源模型 71.6 对 59），引用这类分数必须交代评测主体与条件。

局限说明：本文多处依赖单一来源 - PPT 快照数字（star 数、代码行数、日期均为演讲制作时读数或约数）、直播转写口述（同音噪声大，人名、版本号、项目名多处存疑）与读图近似值（logprob 差、reward 曲线、通过率），已逐处标注置信度与限定条件。第一节 Table 5 仅保留经台账核对的单元格，完整数据请以 OpenRLHF 论文原表为准。第八节全部对应讲者标记为 optional、直播未逐页展开的章节，成熟度有限。涉及第三方的二手说法（字节内部框架使用情况、Cohere 估值等）均为讲者或主持人个人理解，不应作为事实性断言转引。引用本文数据前，请优先核实 vime#11、vime#307、vime#300、vime#242、vLLM#45221 等原始链接（编号可能存在仓库名截断问题），并与讲者确认"qwen3.6"等幻灯片原文拼写。